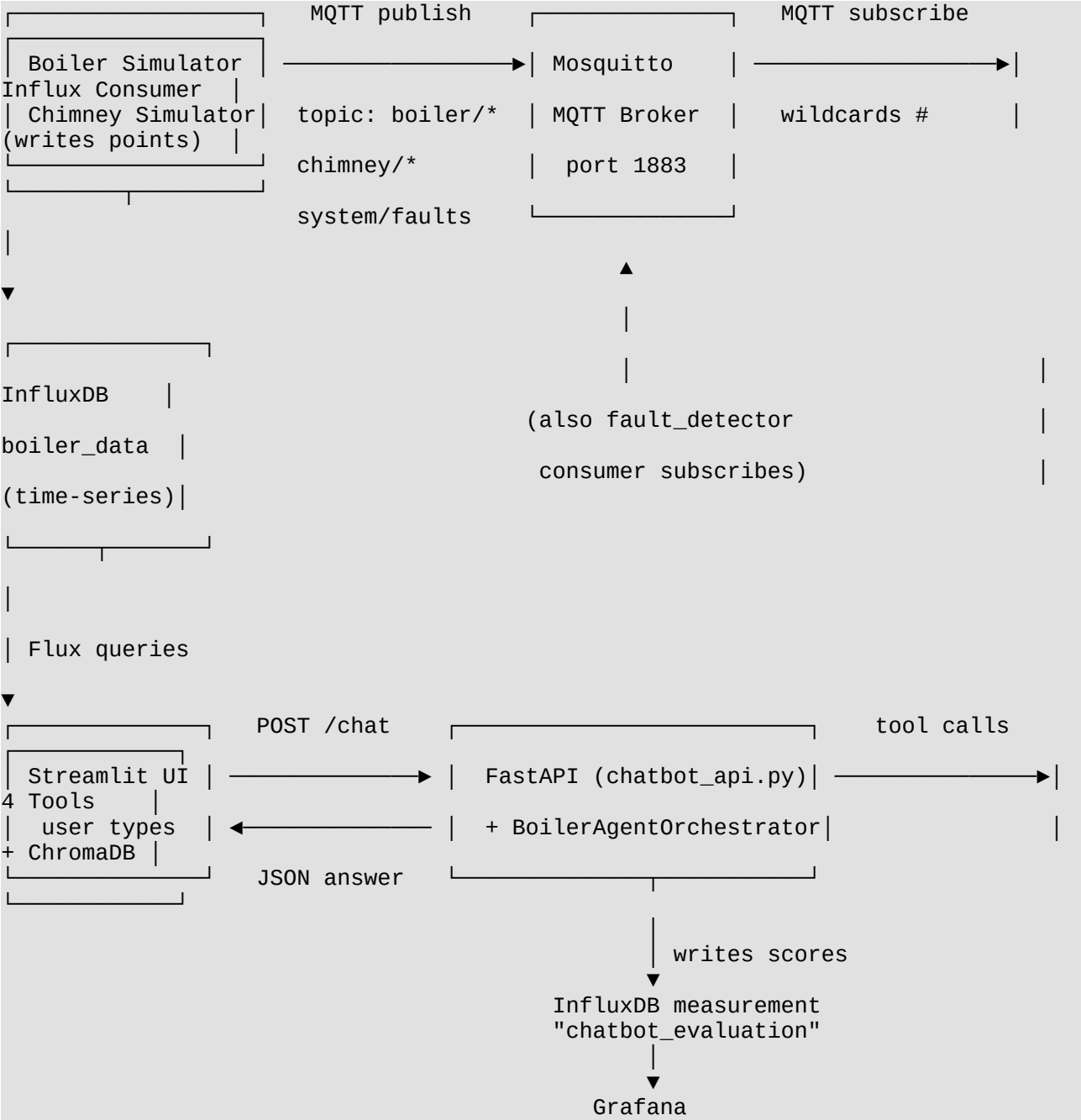


Boiler IoT Agentic RAG — Project Explanation Guide

Audience: Supervisor / reviewer. Purpose: Single document covering architecture, tool calling, ReAct loop, RAGAS math, prediction math, orchestration, memory, end-to-end data flow, and current RAGAS quality issue with fixes.

1. End-to-End Project Flow (Publisher → Subscriber → Agent → User)



Step-by-step

1. **Publisher** — publisher/simulators/boiler_simulator.py +

- `chimney_simulator.py` generate synthetic sensor values (sine-wave normal operation + drift + injected faults). Each reading is serialized to JSON and published to MQTT topics:
- `boiler/<sensor>` (e.g. `boiler/main_steam_temp`)
 - `turbine/<sensor>`
 - `chimney/<sensor>`
 - `system/faults` (QoS 2 — never lose a fault)
2. **MQTT Broker** — Mosquitto running on port 1883 (defined in `docker-compose.yml`). Acts as the pub/sub fan-out.
 3. **Subscribers** (multiple consumers, producer-consumer pattern):
 - `consumers/influx_consumer.py` — subscribes to wildcards `boiler/#`, `chimney/#`, `system/faults`, writes every point into InfluxDB measurements `boiler_sensors`, `chimney_sensors`, `system_faults`.
 - `consumers/fault_detector.py` — classifies readings vs alarm bands and emits structured fault events.
 - `subscriber/boiler_subscriber.py` / `chimney_subscriber.py` — debug listeners.
 4. **InfluxDB** — time-series store with bucket `boiler_data`. All tools query Flux against it.
 5. **FastAPI** (`api/chatbot_api.py`) — exposes `/chat`, `/chat/stream`, `/status`, `/health`. On startup it constructs `BoilerAgentOrchestrator` and `BoilerEvaluator` once (heavy init).
 6. **Orchestrator** (`assistant/agent/orchestrator.py`) — runs the ReAct loop with fine-tuned Gemini and the 4 tools.
 7. **Evaluator** (`evaluation/evaluator.py`) — runs RAGAS on every `/chat` answer, writes scores into Influx measurement `chatbot_evaluation`.
 8. **Streamlit** (`streamlit_app.py`) — chat UI, calls `/chat`, displays answer + tool trace + scores.
 9. **Grafana** — visualises both raw sensor data (boiler health dashboards) and `chatbot_evaluation` (chatbot quality dashboards).
-

2. Tool Calling — How It's Implemented

The agent uses **Vertex AI Gemini Function Calling** (Google's native tool-use API), not LangChain.

2.1 The four tools

Tool	File	Purpose	Backing store
<code>fetch_realtime_</code>	<code>assistant/</code>	Live readings,	InfluxDB (last 5 min)

Tool	File	Purpose	Backing store
sensors	agent/tools/ realtime_tool.py	NORMAL/WARNING /CRITICAL status	
search_knowledge_base	assistant/ agent/tools/ knowledge_tool.py	Semantic search over engineering docs	ChromaDB + OpenAI embeddings
get_fault_history	assistant/ agent/tools/ fault_history.py	Past faults in time window	InfluxDB system_faults
predict_trend	assistant/ agent/tools/ predict_trend.py	Linear regression, time-to-threshold	InfluxDB

2.2 Schemas declared to the model

assistant/agent/tool_schemas.py declares each tool as a `FunctionDeclaration` with name, description, JSON-Schema parameters, and required args. These are bundled into a `Tool` object:

```
BOILER_AGENT_TOOLS = Tool(function_declarations=[
    fetch_realtime_sensors_schema,
    search_knowledge_base_schema,
    get_fault_history_schema,
    predict_trend_schema,
])
```

This `Tool` is passed on every `generate_content(..., tools=[BOILER_AGENT_TOOLS])` call so Gemini knows what it may call.

2.3 Dispatcher

orchestrator.py keeps a name → function registry:

```
TOOL_REGISTRY = {
    "fetch_realtime_sensors": fetch_realtime_sensors,
    "search_knowledge_base": search_knowledge_base,
    "get_fault_history": get_fault_history,
    "predict_trend": predict_trend,
}
```

`execute_tool(name, args)` looks up the function, calls `func(**args)`, and returns its string result (catches `TypeError` for bad args, generic `Exception` for runtime failures — never raises, so the loop is robust).

2.4 Framework choice

- **No LangChain / LlamaIndex.** Tool calling is Vertex AI native

(`vertexai.generative_models`). This was a deliberate choice — less abstraction, easier to debug streams of `Content/Part`, and the fine-tuned Gemini endpoint plugs in directly.

- **ChromaDB** is the vector DB for the knowledge tool, with `text-embedding-3-small` from OpenAI.

3. The ReAct Loop — Explained

ReAct = **R**eason + **A**ct. The model alternates between *thinking* (deciding which tool to call) and *acting* (executing the tool), feeding results back until it has enough to answer.

3.1 What the loop looks like in this project

File: `assistant/agent/orchestrator.py`, method `BoilerAgentOrchestrator.run()` (lines 204–351).

```
messages = [user_question] # initial state

while step_count < MAX_AGENT_STEPS (default ~6):
    response = model.generate_content(messages, tools=[BOILER_AGENT_TOOLS])
    candidate = response.candidates[0]

    tool_calls = parts where function_call exists
    text_parts = parts with text

    if tool_calls:
        for tc in tool_calls:
            result = execute_tool(tc.name, tc.args)
            append tool-result Part
            messages.append( model's tool-call request )
            messages.append( our tool-result Parts )
            # → loop continues, model sees the results next turn

    elif text_parts:
        final_answer = joined text
        break # model produced final answer
    else:
        break # empty response, abort
```

3.2 Why this is a ReAct loop (not single-shot RAG)

Single-shot RAG	This (ReAct)
Embed query → retrieve docs → stuff into prompt → generate	Model decides at every turn whether to fetch live sensors, search KB, fetch fault history, predict, or answer
One tool, one path	Multi-tool, multi-step, parallel calls in same turn possible
No reasoning over intermediate results	Each tool result becomes input to the next reasoning step

Example trace (real run):

```
Step 1: model calls fetch_realtime_sensors() + get_fault_history(60) ← parallel
Step 2: model sees CO is high + recent HIGH_CO fault → calls
search_knowledge_base("why CO high")
Step 3: model produces final answer citing sensor value + KB doc
```

3.3 Safety rails

- `MAX_AGENT_STEPS` caps runaway loops.
 - `MAX_TOKENS` finish reason is surfaced (line 251) so truncated answers aren't silently shipped.
 - Tool errors are returned as strings (never raise) → model can recover or report truthfully.
 - `final_answer is None` fallback (line 332) returns a summary of what was gathered.
-

4. Orchestration — What Happens Behind the Scenes

"Orchestration" here means how a single user question flows through code:

```
User question → /chat (FastAPI)
↓
BoilerAgentOrchestrator.run(question)
  1. system_instruction is preloaded into the GenerativeModel
  2. messages = [Content(role=user, parts=[question])]
  3. ReAct loop (Section 3)
     - model.generate_content(messages, tools=[BOILER_AGENT_TOOLS])
     - if function_call → execute_tool(name, args)
     - append model-turn + tool-result-turn to messages
  4. returns {answer, steps, total_steps, latency_ms, timestamp}
↓
contexts = [f"Tool: {s.tool}\nResult: {s.result_preview}" for s in steps]
tools_called = [s.tool for s in steps]
↓
BoilerEvaluator.evaluate_answer(question, answer, contexts, latency, steps,
tools_called)
  1. Build HuggingFace Dataset({question, answer, contexts})
  2. ragas.evaluate(dataset, metrics=[faithfulness, answer_relevancy],
                    llm=ChatVertexAI(gemini-2.5-flash), embeddings=OpenAI)
  3. df = result.to_pandas() → extract row 0 scores
  4. tool_precision = |called ∩ expected| / |expected|
  5. overall_quality = 0.4·F + 0.4·R + 0.2·TP
  6. write Point("chatbot_evaluation") to InfluxDB
↓
JSON response → Streamlit renders answer + steps + scores
```

Key orchestration details

- **One process, in-memory state.** Agent + evaluator are instantiated at FastAPI startup (module level). No external workflow engine — orchestration is just Python control flow.
- **Streaming variant.** `/chat/stream` uses `run_stream()` which yields SSE events (`tool_start`, `tool_end`, `answer_chunk`, `done`). Evaluation runs after the stream

completes so it still hits Influx.

- **Per-request stateless on the server.** Conversation history is rebuilt each call (see §6 on memory).

5. RAGAS — How Scores Are Calculated

5.1 What the project measures

Metric	Source	What it captures
faithfulness	RAGAS	Is every factual claim in the answer supported by the tool outputs (contexts)?
answer_relevancy	RAGAS	Does the answer actually address the question?
tool_precision	custom	Did the agent call the tools we expected for this question type?
overall_quality	weighted sum	$0.4 \cdot F + 0.4 \cdot R + 0.2 \cdot TP$
latency_ms, steps_taken	timing/loop	Performance

5.2 Faithfulness — the math

RAGAS computes faithfulness with a **2-stage LLM pipeline** (judge = Gemini 2.5 Flash here):

1. **Claim extraction.** Judge LLM reads `answer` and outputs a list of atomic factual claims. Example answer: *"CO is 320 ppm which is critical and rising at 5 ppm/min."* → claims [C1: C0=320 ppm, C2: status=critical, C3: rising 5 ppm/min].
2. **Claim verification.** For every claim C_i , judge LLM is asked: *"Given the contexts, is claim C_i supported? yes/no."*

Score:

```
faithfulness = (# supported claims) / (# total claims)
```

Range [0, 1]. 1.0 = every claim is grounded in tool output. 0.0 = hallucinated.

5.3 Answer Relevancy — the math

This one uses **embeddings**, not just LLM judging:

1. Judge LLM generates N synthetic questions (default 3) that the **answer** would plausibly answer.
2. Embed all of: original question q and each generated question $\hat{q}_i$ (OpenAI text -

embedding-3-small).

3. Compute cosine similarity between q and each $\hat{q}_i$.
4. Score = mean cosine similarity.

```
answer_relevancy = (1/N) ·  $\sum \cos(\text{emb}(q), \text{emb}(\hat{q}_i))$ 
```

Intuition: if the answer's "implied questions" embed close to the user's actual question, the answer was on-topic. Off-topic answers produce divergent reverse-questions → low score.

5.4 Tool Precision — the math (custom)

evaluation/evaluator.py::calculate_tool_precision:

```
expected = lookup(question keywords → expected tools)    # EXPECTED_TOOLS_MAP
called    = tools the agent actually used
tool_precision = |called ∩ expected| / |expected|
```

Keyword heuristic; not a real classifier — that's why its weight is only 0.2.

5.5 Where pandas comes in

result = ragas.evaluate(...) returns a RAGAS EvaluationResult object.

.to_pandas() converts it to a DataFrame with one row per sample and one column per metric — that's why line 191 calls df = result.to_pandas() and then

df["faithfulness"].iloc[0]. The raw == raw trick on line 197 is a NaN check (NaN ≠ NaN in IEEE 754) for when RAGAS fails to score a sample.

6. Short-Term Memory (Conversation Memory)

6.1 Where it lives

This project has **two layers** of short-term memory:

Layer A — Streamlit session (UI side). streamlit_app.py keeps

st.session_state.history (a list of past turns) — survives across reruns but lives only for that browser session.

Layer B — ReAct intra-turn memory (agent side). Inside orchestrator.run(), the messages list grows turn by turn as the agent calls tools and sees results. This is the model's working memory for the single question:

```
messages = [
    Content(role=user,    parts=[question]),
    Content(role=model,   parts=[function_call(...)]), # appended after step 1
    Content(role=user,    parts=[FunctionResponse(...)]), # tool result
    Content(role=model,   parts=[function_call(...)]), # step 2
    ...
    Content(role=model,   parts=[final text answer]),
]
```

6.2 What is *not* yet implemented

- Cross-turn memory on the backend. The `session_id` field exists on `ChatRequest` (line 43 of `chatbot_api.py`) but is **not currently used** to persist or look up prior turns. Each `/chat` call starts the ReAct loop with only the current question. If supervisor asks "follow-up Qs work?" — answer is: only on the Streamlit side, not yet stitched into the agent prompt.
- A dedicated memory store (e.g. Redis / SQLite chat log) is on the roadmap (`PRODUCTION_GAPS.md`).

6.3 How to extend (if asked)

Add to orchestrator: pull prior (`question`, `answer`) pairs for `session_id`, prepend them as alternating `user/model Content` turns before the new question. Trim by token budget.

7. The Math Behind `predict_trend`

File: `assistant/agent/tools/predict_trend.py`.

7.1 Step 1 — pull a series

Flux query: last `window_minutes` of the sensor, downsampled to **1-minute means**:

```
from(bucket: BUCKET)
  |> range(start: -{window_minutes}m)
  |> filter(_measurement == "boiler_sensors")
  |> filter(sensor == "{sensor_name}")
  |> filter(_field == "value")
  |> aggregateWindow(every: 1m, fn: mean, createEmpty: false)
```

Result: `points = [(t0, y0), (t1, y1), ..., (tn-1, yn-1)]` with `ti` spaced 1 minute apart.

7.2 Step 2 — simple linear regression (Ordinary Least Squares)

Treat `xi = i` (the index, also the minute offset). Fit `y = m · x + b` and solve for slope `m`:

$$m = \frac{n \cdot \sum(x_i \cdot y_i) - \sum x_i \cdot \sum y_i}{n \cdot \sum x_i^2 - (\sum x_i)^2}$$

This is exactly what lines 78–87 compute:

```
sum_x = sum(range(n))
sum_y = sum(values)
sum_xy = sum(i * v for i, v in enumerate(values))
sum_xx = sum(i * i for i in range(n))
slope = (n*sum_xy - sum_x*sum_y) / (n*sum_xx - sum_x**2)
```

Since `xi` is in minutes, `slope = rate of change per minute` (units/min).

7.3 Step 3 — time-to-threshold

Get current value y_{n-1} , fetch normal band (lo , hi) from `SENSOR_NORMAL_RANGE`.

- **Rising** ($m > 0$, $current < hi$): $minutes_to_upper = (hi - current) / m$
- **Falling** ($m < 0$, $current > lo$): $minutes_to_lower = (current - lo) / |m|$
- **Stable** ($|m| < 0.01$): no breach in foreseeable horizon.

If breach within 60 min → ⚠ warning. Otherwise .

7.4 Why not ARIMA / Prophet / LSTM?

- Sensor data over 30–60 min is dominated by local linear drift; OLS slope is robust and explainable.
- Tool output must be **fast** (the ReAct loop is calling it under a latency budget) — OLS is $O(n)$ one pass.
- Easy to interpret in the chatbot answer ("current rate of change = +0.4 °C/min").
- More advanced models would be Section-N work — this is documented as a future enhancement in `PRODUCTION_GAPS.md`.

8. Diagnosing Low RAGAS Scores (Faithfulness & Answer Relevancy)

Tool precision is fine → the agent picks the right tools. But faithfulness and relevancy are low. Below is a structured diagnosis with fixes.

8.1 Faithfulness is low — what causes it

Faithfulness drops when **the answer contains claims that aren't in the contexts**. Root causes here:

Cause	Evidence in code	Fix
Truncated context. Each step result is stored as <code>result_preview</code> truncated to 600 chars (<code>orchestrator.py:289</code>). Then <code>/chat</code> builds <code>contexts</code> from <code>result_preview</code> , not the full tool output. The model saw the full result, but RAGAS only sees the truncated version → many real claims look unsupported.	<code>chatbot_api.py:71</code> — <code>contexts.append(f"Tool: {step['tool']}\nResult: {step['result_preview']})"</code>	Store full <code>tool_result</code> separately in <code>steps</code> and feed the full version to RAGAS. Keep <code>result_preview</code> only for the UI.
Model still leans on training	<code>orchestrator.py:74-79</code>	Either (i) stricter prompt: every

Cause	Evidence in code	Fix
memory for IBR / engineering facts the KB doesn't contain. System prompt allows "(b) explicit IBR knowledge you can state with certainty" — those claims won't be in the contexts and RAGAS will mark them unsupported.		factual claim must come from a tool result; or (ii) expand KB so claims are retrievable; or (iii) ignore that class of claim in eval.
Knowledge base is thin / off-topic. If <code>search_knowledge_base</code> returns weakly-relevant docs, the answer cites engineering facts the docs don't actually contain.	<code>knowledge_base/boiler_guide.py</code>	Audit KB coverage of the questions in <code>query.csv</code> . Add docs for missing fault codes, IBR clauses, multi-sensor patterns. Re-index.
Judge LLM is too cheap. Faithfulness depends on <code>gemini-2.5-flash</code> (the judge) correctly identifying claim support. Flash misses subtle paraphrases.	<code>evaluator.py:126</code>	Switch judge to <code>gemini-2.5-pro</code> (or Claude Sonnet) for evaluation. Slower but more accurate scores.
Bad context formatting. Contexts joined as one string per step ("Tool: X\nResult: Y"); RAGAS expects each context as a clean "document". Headers may confuse the judge.	<code>chatbot_api.py:70-72</code>	Strip the Tool: header and pass the raw result; or break each result into chunks.

8.2 Answer relevancy is low — what causes it

Answer relevancy drops when the **reverse-generated questions** drift away from the original. Root causes here:

Cause	Fix
Over-templated answers. The 5-section template (Current Status / Diagnosis / Root Cause / Immediate Actions / Prevention) is applied to questions that didn't ask for all five. The extra sections describe things the user didn't ask → judge generates reverse-questions about <i>those</i> , embedding far from the original.	Tighten prompt — already partially done (lines 142-167) but enforce harder: forbid sections that have no data; "answer proportional to question length".
Generic preamble. Answers start with "Based on the current sensor readings..." then list everything. Reverse-generated questions become generic boiler health questions.	Force the first sentence to directly answer the user's question, then add evidence after.

Cause	Fix
Long answers. Embedding similarity drops as the answer covers many topics — the embedding of the synthetic question is averaged from a wider text → less specific.	Cap answer length for simple questions.
Embedding model mismatch. RAGAS uses OpenAI embeddings; the KB is also indexed with OpenAI — fine. But if the answer language doesn't match the question phrasing style, cosine drops.	Prompt the model to echo the user's terms in the first sentence.

8.3 Quick wins, ranked

1. **Send full tool results to RAGAS** (not the 600-char preview). Single highest-impact fix.
2. **Tighten the answer-shape rule:** short questions get short answers, no template padding.
3. **Upgrade the judge LLM** from Flash to Pro/Sonnet.
4. **Audit + extend ChromaDB knowledge base** so factual claims are retrievable.
5. **Strip "Tool:" headers from contexts** passed to RAGAS.
6. **Re-evaluate:** re-run `query.csv` after each change, compare distributions in Grafana.

8.4 Verification dashboard

In Grafana (Flux on the `chatbot_evaluation` measurement) plot the per-metric distribution before vs after each change. Pivot query (already in `evaluation` queries) gives you per-question scores so you can see which question types regress.

9. Quick Reference — Where Each Concept Lives

Concept	File
MQTT publishers	<code>publisher/simulators/*.py</code>
MQTT → InfluxDB	<code>consumers/influx_consumer.py</code>
Fault detection	<code>consumers/fault_detector.py</code>
Tool schemas (Gemini)	<code>assistant/agent/tool_schemas.py</code>
Tool implementations	<code>assistant/agent/tools/*.py</code>
ReAct loop + orchestration	<code>assistant/agent/orchestrator.py</code>
RAGAS evaluator	<code>evaluation/evaluator.py</code>
API surface	<code>api/chatbot_api.py</code>
UI / short-term memory (UI)	<code>streamlit_app.py</code>
Config (Influx, Vertex, etc.)	<code>assistant/config.py</code>
KB documents	<code>knowledge_base/boiler_guide.py</code>

Concept	File
KB indexer (Chroma)	knowledge_base/indexer.py

10. One-Slide Summary (for the supervisor meeting)

- **Architecture:** MQTT pub/sub → InfluxDB time series → FastAPI + Vertex AI Gemini (fine-tuned) → 4 tools → RAGAS eval → InfluxDB → Grafana.
- **Tool calling:** native Vertex AI function calling (no LangChain). 4 tools wired through a TOOL_REGISTRY dispatcher.
- **ReAct loop:** messages list grown turn by turn; model alternates `function_call` and final text; bounded by MAX_AGENT_STEPS.
- **RAGAS:** faithfulness = supported-claims / total-claims; answer_relevancy = mean cosine between user Q and reverse-generated Qs; both judged by Gemini 2.5 Flash with OpenAI embeddings.
- **Prediction:** OLS slope over last N minutes of sensor data → time-to-threshold.
- **Memory:** per-question working memory in the ReAct messages; per-session UI history in Streamlit; no server-side cross-turn memory yet.
- **Quality problem:** faithfulness/relevancy low because (i) we send a 600-char preview of tool output to RAGAS instead of the full result, (ii) answer template is too rigid, (iii) judge LLM is the cheapest tier. Fixes ranked in §8.3.